

EXPERIMENT 22

Title: Write a Prolog Program to Print Whether a Particular Bird Can Fly or Not. Incorporate Required Queries.

Aim

To develop a Prolog program to determine whether a particular bird can fly or not using facts, rules, and logical queries.

Objective

- To understand the use of facts and rules in Prolog.
- To represent properties of different birds.
- To determine whether a particular bird can fly.
- To understand logical inference using Prolog.
- To practice querying a knowledge base.

Theory

Prolog can be used to represent knowledge using facts and rules. In this experiment, facts about different birds are stored in a knowledge base. A rule is then used to determine whether a particular bird can fly.

Most birds can fly, but some birds such as penguins, ostriches, and emus cannot fly. Prolog uses the available facts and rules to determine the result of a query.

For example:

```
bird(parrot).
```

```
can_fly(parrot).
```

indicates that a parrot is a bird and can fly.

Similarly:

```
bird(penguin).
```

```
cannot_fly(penguin).
```

indicates that a penguin is a bird but cannot fly.

Algorithm

Step 1: Start the program.

Step 2: Define facts representing different birds.

Step 3: Define facts for birds that can fly.

Step 4: Define facts for birds that cannot fly.

Step 5: Define rules to determine whether a bird can fly.

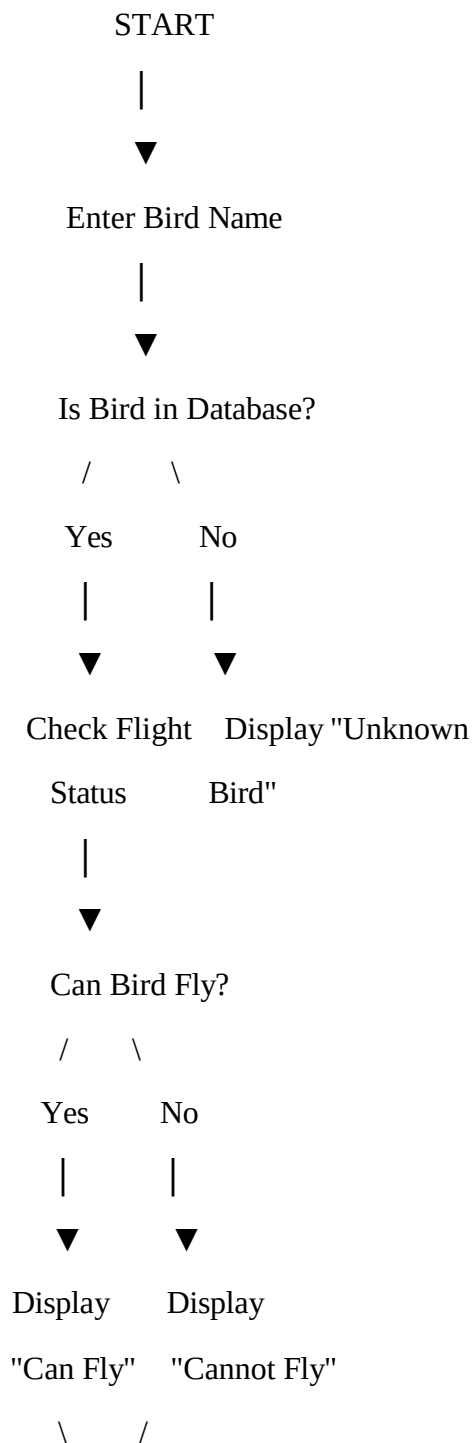
Step 6: Enter the name of a bird as a query.

Step 7: Match the query with the available facts.

Step 8: Display whether the particular bird can fly or not.

Step 9: Stop the program.

Flowchart



\ /



STOP

Prolog Program

% Bird Database

bird(parrot).

bird(eagle).

bird(sparrow).

bird(pigeon).

bird(penguin).

bird(ostrich).

bird(emu).

% Birds that can fly

can_fly(parrot).

can_fly(eagle).

can_fly(sparrow).

can_fly(pigeon).

% Birds that cannot fly

cannot_fly(penguin).

cannot_fly(ostrich).

cannot_fly(emu).

% Rules

flies(Bird) :-

bird(Bird),
can_fly(Bird).

does_not_fly(Bird) :-

bird(Bird),
cannot_fly(Bird).

Queries

Query 1: Check whether a parrot can fly

?- flies(parrot).

Sample Output

true.

Query 2: Check whether a penguin can fly

?- flies(penguin).

Sample Output

false.

Query 3: Check whether a penguin cannot fly

?- does_not_fly(penguin).

Sample Output

true.

Query 4: Find all birds that can fly

?- flies(Bird).

Sample Output

Bird = parrot ;

Bird = eagle ;

Bird = sparrow ;

Bird = pigeon ;

false.

Query 5: Find all birds that cannot fly

?- does_not_fly(Bird).

Sample Output

Bird = penguin ;

Bird = ostrich ;

Bird = emu ;

false.

Explanation

The predicate bird/1 stores the names of birds.

The predicate can_fly/1 stores birds that are capable of flying, while cannot_fly/1 stores birds that cannot fly.

The rule:

flies(Bird) :-

bird(Bird),

can_fly(Bird).

checks whether the given bird is present in the bird database and whether it belongs to the list of flying birds.

Similarly:

does_not_fly(Bird) :-

bird(Bird),

cannot_fly(Bird).

checks whether the bird belongs to the list of birds that cannot fly.

Thus, Prolog can determine the flying ability of a particular bird based on the stored facts.

Advantages

- Simple representation of knowledge using facts.
- Easy to query different birds.

- Demonstrates logical reasoning.
- Easy to add new birds and properties.
- Useful for understanding rule-based systems.

Applications

- Expert systems.
- Knowledge-based systems.
- Artificial Intelligence applications.
- Rule-based classification.
- Decision-making systems.
- Educational AI applications.

Result

The Prolog program was successfully implemented to determine whether a particular bird can fly or not. Required queries were executed successfully and the correct results were obtained.

Conclusion

Thus, the bird flight classification problem was successfully implemented using Prolog. The experiment demonstrates how facts, rules, and queries can be used to represent knowledge and perform logical reasoning in Artificial Intelligence.

```
PS C:\Users\Varshini M> python "C:\Users\Va
--- BIRD DATABASE ---
1. Check whether a bird can fly
2. Display birds that can fly
3. Display birds that cannot fly
4. Exit
Enter your choice: 1
Enter the bird name: eagle
Eagle can fly.

--- BIRD DATABASE ---
1. Check whether a bird can fly
2. Display birds that can fly
3. Display birds that cannot fly
4. Exit
Enter your choice: []
```

```
32     else:
33         print("Bird not found in the database.")
34
35
36 # Main program
37 while True:
38     print("\n--- BIRD DATABASE ---")
39     print("1. Check whether a bird can fly")
40     print("2. Display birds that can fly")
41     print("3. Display birds that cannot fly")
42     print("4. Exit")
43
44     choice = input("Enter your choice: ")
45
46     if choice == "1":
47         bird = input("Enter the bird name: ")
48         check_flying(bird)
49
50     elif choice == "2":
51         print("\nBirds that can fly:")
52         for bird in can_fly:
53             print("-", bird)
54
55     elif choice == "3":
56         print("\nBirds that cannot fly:")
57         for bird in cannot_fly:
58             print("-", bird)
59
60     elif choice == "4":
61         print("Program terminated.")
62         break
63
64     else:
65         print("Invalid choice.")
```

```
1  # BIRD FLYING DATABASE
2
3  # Birds that can fly
4  can_fly = [
5      "Eagle",
6      "Parrot",
7      "Sparrow",
8      "Pigeon",
9      "Crow",
10     "Peacock"
11 ]
12
13 # Birds that cannot fly
14 cannot_fly = [
15     "Penguin",
16     "Ostrich",
17     "Emu",
18     "Kiwi"
19 ]
20
21
22 # Function to check whether a bird can fly
23 def check_flying(bird):
24     bird = bird.capitalize()
25
26     if bird in can_fly:
27         print(bird, "can fly.")
28
29     elif bird in cannot_fly:
30         print(bird, "cannot fly.")
31
```